МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Редакционное расстояние

Студент гр. 4343 _____ Семенчик В.В.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2026

Выданное задание.

Написать программу, выполняющую три задачи:

1. Нахождение минимальной стоимости преобразования строки A к строке B с заданной стоимостью операций
2. Вывод последовательности операций
3. Вывод минимального количества преобразований без учёта стоимости

Операции – вставка, замена, удаление, двойная вставка (в начале строки; если первый из вставляемых символов равен предшествующему символу получаемой строки)

Цель работы.

Научиться работать с редакционным расстоянием, закрепить основы динамического программирования.

Динамическое программирование. Описание алгоритма.

Создаётся матрица dp, где dp[i][j] – минимальная стоимость преобразования первых i символов слова A к первым j символам слова B. Изначально первая строка и первый столбец – цена преобразования пустой строки к полной B (только вставка) и строки A в пустой строке (только удаление). Таким образом, операция вставки – это горизонтальное движение по таблице dp, ведь для соответствия i символов к j символам нужно взять известное решение «слева» - преобразование i символов к j-1, и потом просто прибавить недостающий символ. Удаление же – вертикальное движение, то есть нужно взять известное решение преобразования i-1 символов к j символам и удалить лишний символ. dp[0][2] изначально равняется цене двойной вставки, ведь эта операция выполняется в двух случаях: в начале строки (dp[0][2]) и если предшествующий символ получаемой строки равен первому вставляемому символу. Для выполнения операции двойной вставки после проверки условия необходимо взять известное решение dp[i][j-2] и прибавить два символа. Операции замены и соответствия – по диагонали слева-сверху, то есть мы берём известное решение dp[i-1][j-1] и, если символы совпадают, стоимость не увеличивается, а если отличаются, то просто заменяет символ и dp[i][j] = dp[i-1][j-1] + цена замены.

Исследование сложности.

Алгоритм перебирает все возможные комбинации i и j, то есть проходит по всем ячейкам dp, которых i*j, следовательно сложность $O(n * m)$. График подтверждает теоретический вывод.

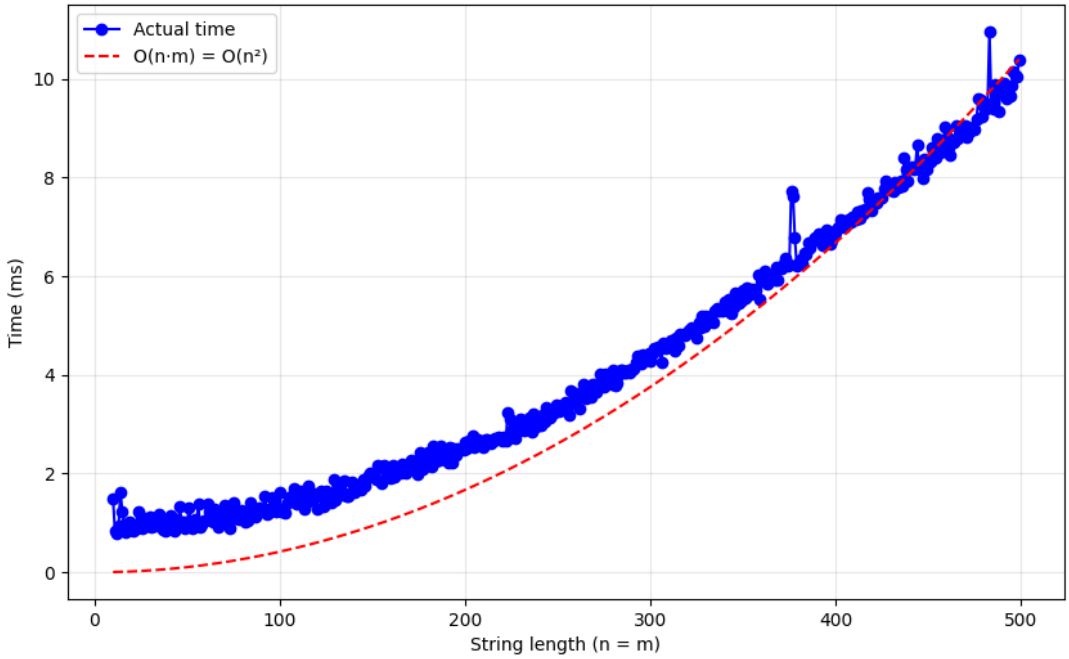

Рисунок 1. Исследование сложности алгоритма

Пространственная сложность – $O(n * m)$, что соответствует размеру op и dp (в нотации не учитывается константа 2).

Выводы.

В ходе лабораторной работы была решена задача о редакционном расстоянии и закреплены принципы динамического программирования.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#define LOG false

#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<vector<int>> graph(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> graph[i][j];
        }
    }

    vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX));

    vector<vector<int>> parent(1 << n, vector<int>(n, -1));

    dp[1 << 0][0] = 0;

    for (int mask = 1; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;

        if (LOG) {
            cout << "Обработка маски: " << mask << " (";
            for (int i = 0; i < n; i++) {
```

```
                if (mask & (1 << i)) cout << i << " ";
            }
            cout << ")" << endl;
        }

        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u))) continue;
            if (dp[mask][u] == INT_MAX) continue;

            if (LOG) cout << "Текущий город u=" << u << ", стоимость
пути: " << dp[mask][u] << endl;

            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                if (graph[u][v] == 0) continue;

                int newMask = mask | (1 << v);
                int newCost = dp[mask][u] + graph[u][v];

                if (LOG) cout << "Рассматриваем переход из u=" << u
<< " в v=" << v
                              << ", вес=" << graph[u][v]
                              << ", новая маска=" << newMask
                              << ", новая стоимость=" << newCost <<
endl;

                if (newCost < dp[newMask][v]) {
                    dp[newMask][v] = newCost;
                    parent[newMask][v] = u;

                    if (LOG) cout << "Обновляем: dp[" << newMask <<
"][" << v
                                  << "]. было " << dp[newMask][v]
                                  << ", стало " << newCost << endl;
                    dp[newMask][v] = newCost;
                    parent[newMask][v] = u;
```

```
                    } else if (LOG) {
                        cout << "Не обновляем: " << newCost << " >= " <<
dp[newMask][v] << endl;
                    }
                }
            }
        }

        int fullMask = (1 << n) - 1;
        int minCost = INT_MAX;
        int lastCity = -1;

        for (int i = 1; i < n; i++) {
            if (dp[fullMask][i] == INT_MAX) continue;
            if (graph[i][0] == 0) continue;

            int totalCost = dp[fullMask][i] + graph[i][0];
            if (totalCost < minCost) {
                minCost = totalCost;
                lastCity = i;
            }
        }

        if (minCost == INT_MAX) {
            cout << "no path" << endl;
            return 0;
        }

        vector<int> path;
        path.push_back(0);
        int mask = fullMask;
        int current = lastCity;

        while (current != 0) {
            path.push_back(current);
```

```
            int prev = parent[mask][current];
            mask &= ~(1 << current);
            current = prev;
        }
        path.push_back(0);

        reverse(path.begin(), path.end());

        cout << minCost << endl;
        for (int i = 0; i < path.size(); i++) {
            cout << path[i];
            if (i < path.size() - 1) cout << " ";
        }
        cout << endl;

        return 0;
}
```

Название файла: graphics.py

```
import subprocess
import time
import matplotlib.pyplot as plt
import random

def run_test(exe, n):
    mat = [[random.randint(1, 100) if i != j else 0 for j in range(n)] for
i in range(n)]
    inp = f"{n}\n" + "\n".join(" ".join(map(str, row)) for row in mat)
    start = time.time()
    subprocess.run(exe, input=inp, text=True, capture_output=True)
    return (time.time() - start) * 1000

sizes = list(range(3, 21))
t_main, t_alsh = [], []

for n in sizes:
    tm = [run_test("./main", n) for _ in range(3)]
    ta = [run_test("./alsh", n) for _ in range(3)]
    t_main.append(sum(tm) / len(tm))
    t_alsh.append(sum(ta) / len(ta))

scale_main = t_main[-1] / ((2 ** sizes[-1]) * (sizes[-1] ** 2))
scale_alsh = t_alsh[-1] / (sizes[-1] ** 4)
theor_main = [scale_main * (2 ** n) * (n ** 2) for n in sizes]
theor_alsh = [scale_alsh * (n ** 4) for n in sizes]

plt.figure(figsize=(10, 6))
```

```
plt.plot(sizes, t_main, 'bo-', label='./main')
plt.plot(sizes, theor_main, 'r--', label='O(n²·2ⁿ)')
plt.xlabel('n'); plt.ylabel('мс'); plt.legend(); plt.grid(True, alpha=0.3)
plt.savefig('graph1.png', dpi=100)

plt.figure(figsize=(10, 6))
plt.plot(sizes, t_alsh, 'go-', label='./alsh')
plt.plot(sizes, theor_alsh, 'r--', label='O(n⁴)')
plt.xlabel('n'); plt.ylabel('мс'); plt.legend(); plt.grid(True, alpha=0.3)
plt.savefig('graph2.png', dpi=100)
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ ПРОГРАММЫ

Входные данные	Выходные данные
1 1 1 1 a a	M 0
1 1 1 1 ABC ABC	MMM 0
1 1 1 1 abc def	RRR 3
1 2 3 4 abc ac	MDM 3
5 1 1 2 hello hllo	MDMMM 1
1 10 1 5 world word	MMMDM 1
1 1 1 1 abc aabbcc	MPMIM 2
1 1 1 1 a aa	IM 1